Pontificia Universidad JAVERIANA Colombia

Orchestrator-workers

Semana 10 · Delegación dinámica y subgrafos
Tópicos Avanzados en Analítica II – IA Generativa
Maestría en Analítica para la Inteligencia de Negocios

Sergio Alberto Mora Pardo - H1 2027
GitHub · LinkedIn · Repositorio del curso

Tópicos Avanzados en Analítica II

Estructura del curso

Fundamentos con LangChain
Semanas 1–3 ✓

RAG y evaluación
Semanas 4–6 ✓

Model Context Protocol
Semana 7 ✓

LangGraph y patrones agénticos
Semanas 8–11

LLMOps y despliegue
Semana 12

Semana 10 · Orchestrator-workers y subgrafos

- Cuando no sabemos cuántas tareas hay
- `Send`: un nodo por elemento
- Cada trabajador con su propia entrada
- El trabajador como subgrafo
- Planear, delegar y sintetizar
- Paralelización fija frente a orquestación

Las ideas que traemos de las semanas 8 y 9

Repaso

Antes de avanzar

1. En la paralelización de L19 los revisores eran fijos: siempre tres.
2. Un reducer junta las salidas de las ramas sin que se pisen.
3. Un grafo puede pausarse y continuar con el estado guardado.

Un ticket con varias preguntas

El problema de la semana

La división depende de la entrada

«Soy C-3307 y tengo 75 días de mora. ¿Cuánto me pueden **ampliar el plazo** si reestructuro, si ya me **reportaron** a centrales de riesgo y si puedo **pagar por PSE** desde otro banco?»

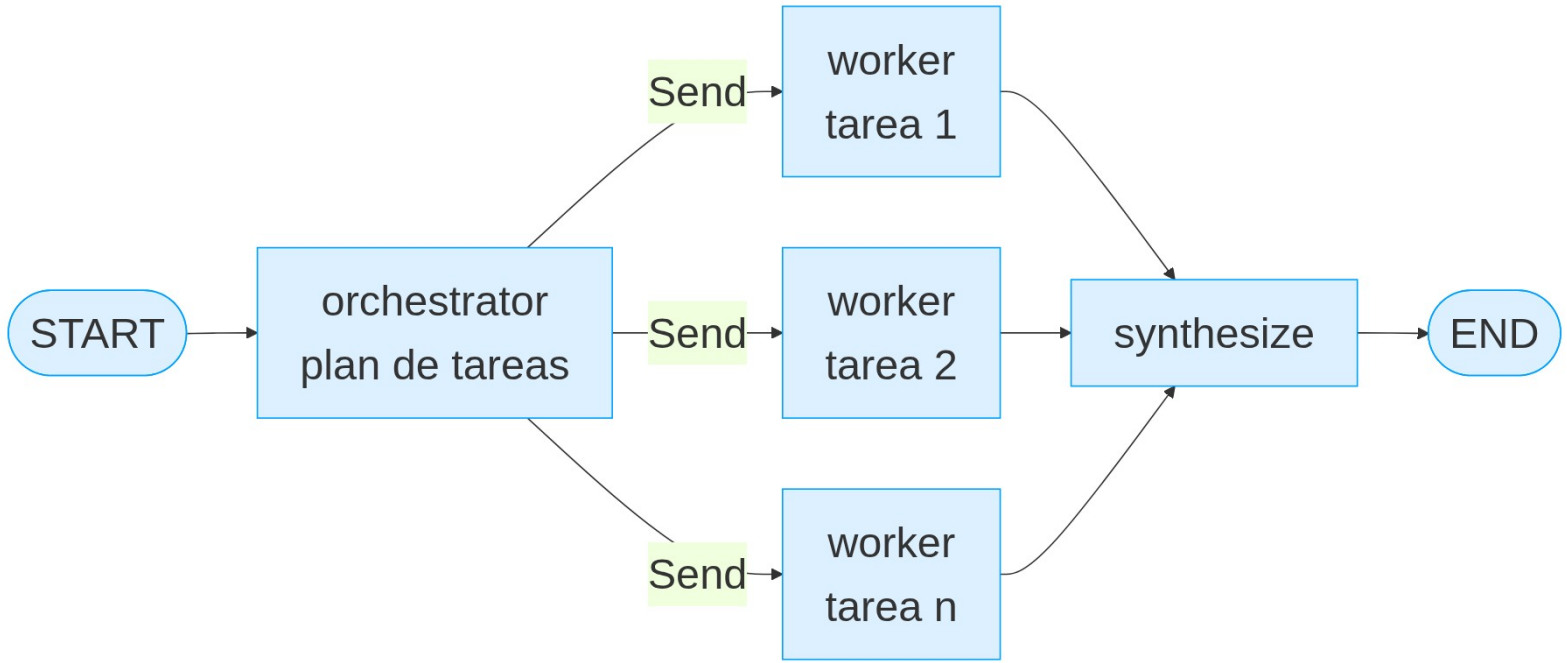
Otro cliente escribe una sola pregunta. Otro, cinco.

Planear, delegar y sintetizar

L21 · sección 3 · unas 5 llamadas

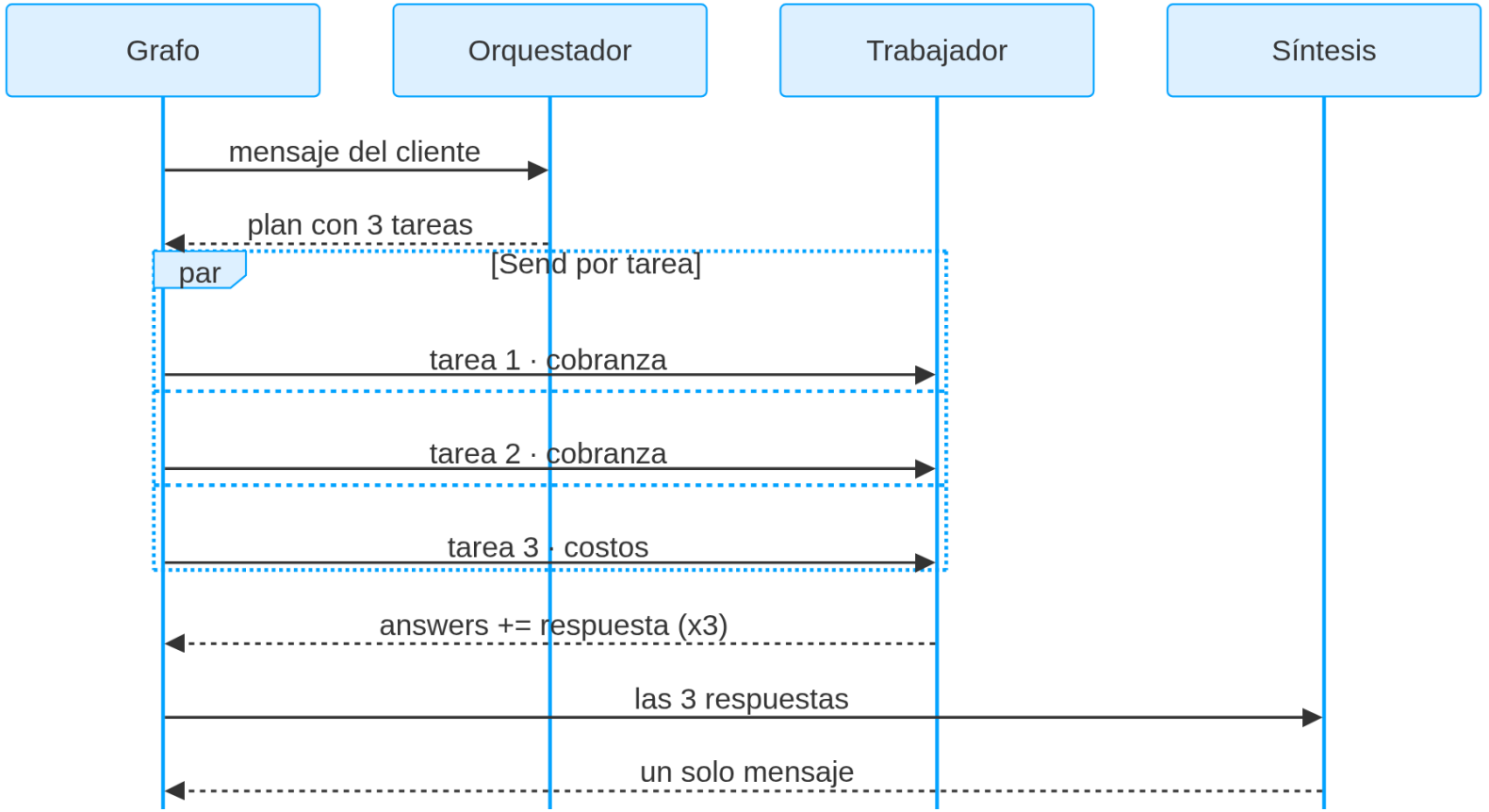
1 + N + 1 llamadas

El orquestador decide cuántas tareas hay; Send lanza un trabajador por tarea.



Send en el tiempo

L21 · diagrama de secuencia



1 + N + 1 llamadas. El plan decide cuántos trabajadores corren; los tres corren a la vez y cada uno agrega su respuesta a `answers`.

Send: un nodo por elemento

L21 · sección 1 · sin modelo

Fan-out dinámico

Cada Send lleva su propia entrada, distinta del estado principal.

Cómo funciona

1. El orquestador deja en el estado una lista de tareas.

2. Send lanza un trabajador por tarea, cada uno con su propia entrada.

3. Un reductor junta las respuestas; un plan vacío va directo a la síntesis.

Un plan con salida estructurada

L21 · sección 3

El orquestador

La política elegida para cada tarea define su contexto.

Cómo funciona

1. Cada tarea es una pregunta del ticket y la política que la responde.

2. El modelo devuelve el plan con salida estructurada: una tarea por pregunta distinta.

3. La política elegida decide qué documentos recibe cada trabajador.

El trabajador como subgrafo

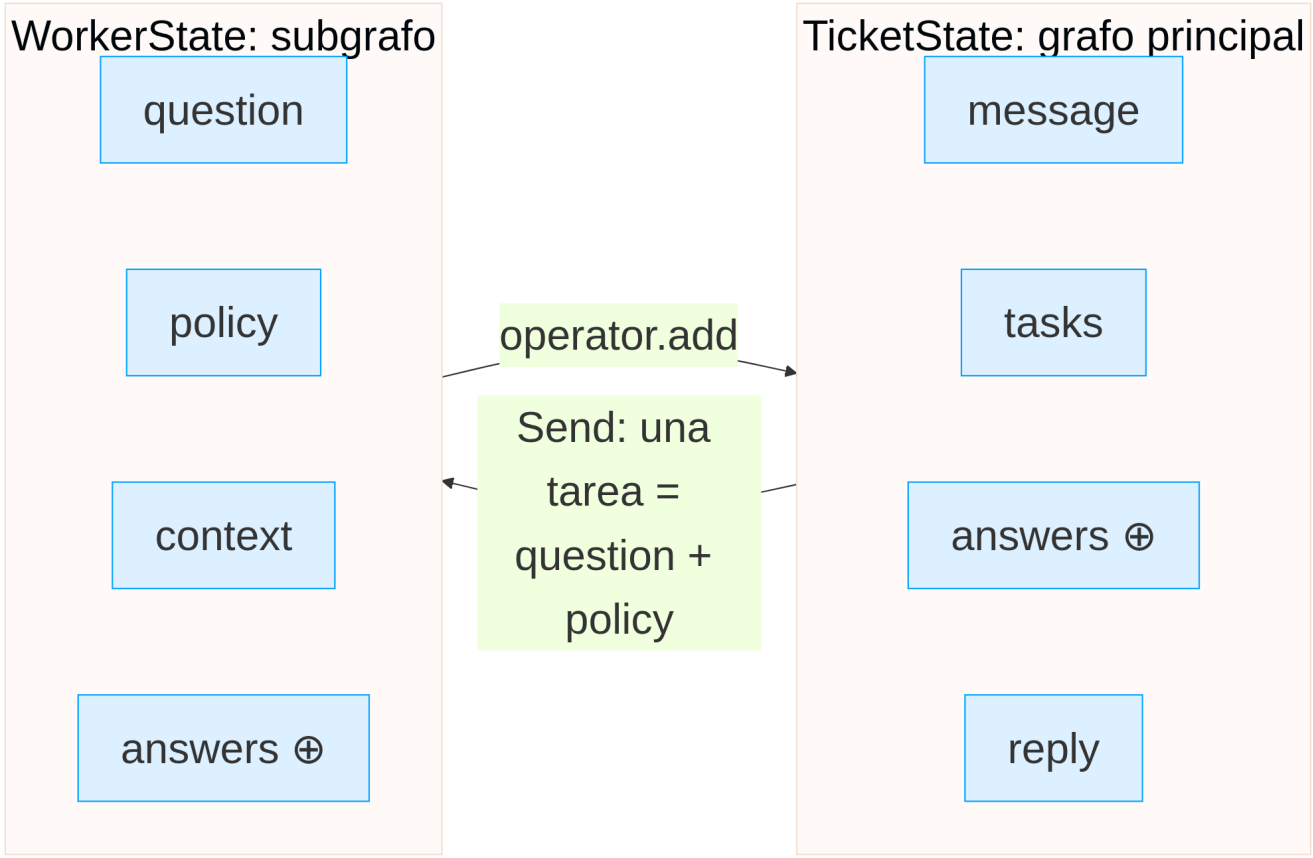
L21 · sección 2

Una pieza reutilizable

1. `WorkerState` tiene sus propios campos: pregunta, política y contexto.
2. El subgrafo carga la política y responde con citas.
3. Comparte con el grafo principal solo la lista `answers`.
4. Se puede probar por separado, con una entrada fija.

Dos estados, una llave compartida

L21 · sección 2



Estado propio. Send le entrega a cada trabajador su tarea; los dos estados comparten solo `answers`, que se junta con `operator.add`.

Leer el plan

L21 · ejecución guardada

Un plan mal dividido

1. El plan separó las tres preguntas del cliente: bien.
2. Pero asignó la ampliación de plazo a la política de **crédito**; la regla está en **cobranza**.
3. Un trabajador con la política equivocada solo puede decir «no está».
4. Revisar el plan es parte de evaluar este patrón.

Cuándo usar cada patrón

L21 · sección 4

	Paralelización (L19)	Orchestrator-workers (L21)
Número de tareas	Fijo, en el código	Lo decide el modelo
Entrada de cada rama	El estado completo	La suya, con `Send`
Llamadas	Siempre las mismas	Variables: 1 + N + 1
Riesgo principal	Pagar ramas innecesarias	Un plan mal dividido

Vocabulario multiagente

Qué debe quedar explícito en cada término

Término	Qué significa aquí
Orquestador o supervisor	El modelo o nodo que divide el trabajo y decide quién lo hace
Trabajador o subagente	Un nodo o subgrafo con una tarea, su entrada y sus herramientas
Subgrafo	Un grafo compilado usado como nodo, con su propio estado y contrato
Map-reduce con `Send`	Una rama por elemento y un reducer que junta los resultados
Handoff	Pasar la conversación completa a otro agente (no lo usamos en el curso)
Sistema multiagente	Varios agentes coordinados: decir cuál patrón y quién decide

Pontificia Universidad JAVERIANA Colombia

Laboratorio

L21-Orchestrator

¿Qué haremos?

Laboratorio

Nuestro recorrido

1. Lanzar trabajadores con Send a partir de un plan fijo (sin modelo).
2. Escribir el trabajador como subgrafo y probarlo solo.
3. Ejecutar orquestador, trabajadores y síntesis sobre un ticket real.
4. Leer el plan y decidir si estaba bien dividido.

Las ideas que nos llevamos

Cierre

Criterios de construcción

1. Usar orquestación solo cuando las subtareas dependen de la entrada.
2. Send da a cada trabajador su propia entrada.
3. Un subgrafo es una pieza con contrato propio, que se prueba aparte.
4. El plan es un punto de fallo: hay que poder leerlo.

Para seguir leyendo

Lecturas de la semana

Fuente	Qué revisar
Anthropic, Building effective agents	Orchestrator-workers
Documentación de LangGraph	`Send`, map-reduce y subgrafos
Building Applications with AI Agents	Sistemas multiagente y coordinación

Pontificia Universidad JAVERIANA Colombia

Próxima semana

Evaluator-optimizer: corregir una respuesta hasta que pase la rúbrica.
Notebook: `L22-EvaluatorOptimizer.ipynb`

Sergio Alberto Mora Pardo - H1 2027
GitHub · LinkedIn · Repositorio del curso